



Capítulo 2 - Estruturas condicionais e de repetição

Tomar decisões e repetir tarefas são recursos essenciais para organizar a automação da ferrovia.

Introdução	No capítulo anterior, organizamos instruções em funções. Agora veremos como escolher quais instruções executar e como repetir uma tarefa de maneira controlada.
Objetivo	Utilizar estruturas condicionais e de repetição em C++, compreendendo quando cada uma é adequada.
Meta de aprendizagem	Ao final, você deverá conseguir escrever uma decisão com if e else, escolher entre opções com switch e realizar repetições com for e while.

1. Decisões no programa

Uma estrutura condicional permite executar instruções conforme uma condição. Essa condição é uma expressão cujo resultado pode ser verdadeiro ou falso.

Na ferrovia, uma condição pode representar um botão pressionado, um trecho ocupado ou uma rota selecionada. O programa utiliza essa informação para escolher uma ação.

Exemplo didático: se um trecho estiver ocupado, mostrar um aviso. Caso contrário, mostrar que ele está livre. Neste capítulo, vamos simular as informações com variáveis, sem conectar componentes.

2. A estrutura if e else

if significa “se”. O bloco de instruções entre chaves é executado quando a condição é verdadeira. else significa “caso contrário” e permite executar outro bloco quando a condição é falsa.

```
bool trackOccupied = true;

if (trackOccupied) {
    Serial.println("Track occupied");
} else {
    Serial.println("Track clear");
}
```

A variável trackOccupied representa “trecho ocupado”. O tipo bool guarda true (verdadeiro) ou false (falso).

Com o valor true, o programa escreve Track occupied. Ao trocar para false, escreve Track clear. A parte else é opcional: podemos escrever apenas um if quando não houver uma ação alternativa.

Comparações



Operador	Significado	Exemplo
==	Igual a	routeNumber == 1
!=	Diferente de	routeNumber != 1
>	Maior que	counter > 3
<	Menor que	counter < 3
>=	Maior ou igual a	counter >= 3
<=	Menor ou igual a	counter <= 3

Não confunda = com ==. Um sinal de igual atribui um valor à variável. Dois sinais de igual comparam valores.

3. Mais de duas possibilidades: else if

Quando houver várias condições, podemos acrescentar else if, que significa “caso contrário, se”.

```
int routeNumber = 2;

if (routeNumber == 1) {
    Serial.println("Route 1 selected");
} else if (routeNumber == 2) {
    Serial.println("Route 2 selected");
} else {
    Serial.println("Unknown route");
}
```

As condições são verificadas na ordem em que aparecem. Quando uma delas é verdadeira, seu bloco é executado e as demais alternativas dessa sequência são ignoradas.

Com routeNumber igual a 2, a mensagem será Route 2 selected.

4. Combinando condições

Uma decisão pode depender de mais de uma informação. Para isso, utilizamos operadores lógicos:

Operador	Significado
&&	E: as duas condições precisam ser verdadeiras.
	OU: pelo menos uma condição precisa ser verdadeira.
!	NÃO: inverte o resultado lógico.



```
bool trackOccupied = false;
bool emergencyActive = false;

if (!trackOccupied && !emergencyActive) {
    Serial.println("Simulation: conditions satisfied");
} else {
    Serial.println("Simulation: operation blocked");
}
```

!trackOccupied significa “trecho não ocupado”. A mensagem de condições atendidas aparece somente quando o trecho está livre e a emergência não está ativa.

Este exemplo ensina a combinação de condições. Uma operação real exige outras verificações, que serão desenvolvidas nos capítulos sobre rotas e segurança. As mensagens não autorizam nem comandam movimentos na ferrovia.

5. Escolhendo opções com switch

switch é útil quando uma variável inteira pode assumir diferentes valores conhecidos. Cada case representa uma opção.

```
int routeNumber = 2;

switch (routeNumber) {
    case 1:
        Serial.println("Route 1 selected");
        break;

    case 2:
        Serial.println("Route 2 selected");
        break;

    default:
        Serial.println("Unknown route");
        break;
}
```

O break encerra o switch após executar a opção escolhida. Sem ele, a execução pode continuar nas instruções dos próximos casos. default trata os valores que não correspondem às opções previstas.

Para comparar intervalos ou combinar condições, if costuma ser mais adequado. Para escolher entre valores inteiros específicos, switch pode tornar a leitura mais clara.

6. Repetindo um número definido de vezes: for

Uma estrutura de repetição executa um bloco de instruções várias vezes. O for é útil quando conhecemos a quantidade de repetições.

```
for (int cycle = 1; cycle <= 3; cycle++) {
    Serial.print("Test cycle: ");
    Serial.println(cycle);
}
```



Parte do código	Função
<code>int cycle = 1</code>	Cria o contador e define seu valor inicial.
<code>cycle <= 3</code>	Permite continuar enquanto o contador for menor ou igual a 3.
<code>cycle++</code>	Acrescenta 1 ao contador após cada execução do bloco.

A condição é verificada antes de cada repetição. O exemplo escreve os números 1, 2 e 3. Quando o contador chega a 4, a condição é falsa e a repetição termina.

Na ferrovia, essa estrutura pode ajudar a organizar uma sequência de testes. Neste exemplo, ela apenas apresenta mensagens; nenhum desvio é movimentado.

7. Repetindo enquanto uma condição for verdadeira: while

while significa “enquanto”. O bloco é repetido enquanto a condição for verdadeira.

```
int counter = 1;

while (counter <= 3) {
  Serial.print("Reading number: ");
  Serial.println(counter);
  counter++;
}
```

A condição é verificada antes de cada execução. Se ela já for falsa na primeira verificação, o bloco não será executado.

Neste exemplo, `counter++` faz a condição mudar até a repetição terminar. Se essa instrução for retirada, o contador permanecerá em 1 e o programa ficará preso na repetição.

A variação do ... while

Na estrutura do ... while, a condição é verificada depois do bloco. Por isso, as instruções são executadas pelo menos uma vez.

```
int counter = 1;

do {
  Serial.println(counter);
  counter++;
} while (counter <= 3);
```

Observe o ponto e vírgula depois de while (...) nessa estrutura.

Na automação, evite deixar o programa esperando indefinidamente dentro de uma repetição. Enquanto esse trecho está executando, as instruções seguintes de `loop()` não são alcançadas. Para acompanhar sensores continuamente, prefira verificações curtas a cada passagem por `loop()`,



usando `millis()` quando precisar controlar o tempo.

8. Diferença entre `loop()` e uma repetição interna

`loop()` é uma função chamada repetidamente pelo ambiente Arduino. Já `for` e `while` são estruturas que repetem um bloco dentro da execução atual.

Se um `for` de três repetições estiver dentro de `setup()`, será executado apenas na inicialização. Se estiver dentro de `loop()`, as três repetições ocorrerão novamente a cada chamada dessa função.

Um `break` também pode encerrar antecipadamente um `for` ou um `while`. Ele sai da repetição mais próxima, sem encerrar todo o programa.

9. Programa completo para testar

Vamos reunir funções, decisões e repetições em um programa simples. Utilize somente uma ESP32 de bancada conectada ao computador por USB. Não são necessários sensores, LEDs ou servos.



```
void showTrackStatus(bool trackOccupied) {
    if (trackOccupied) {
        Serial.println("Track occupied");
    } else {
        Serial.println("Track clear");
    }
}

void showRoute(int routeNumber) {
    switch (routeNumber) {
        case 1:
            Serial.println("Route 1 selected");
            break;

        case 2:
            Serial.println("Route 2 selected");
            break;

        default:
            Serial.println("Unknown route");
            break;
    }
}

void runTestCycles(int totalCycles) {
    for (int cycle = 1; cycle <= totalCycles; cycle++) {
        Serial.print("Test cycle: ");
        Serial.println(cycle);
    }
}

void setup() {
    Serial.begin(115200);
    delay(1000);

    Serial.println("EFMN 74 - Decisions and loops");

    showTrackStatus(true);
    showTrackStatus(false);
    showRoute(2);
    runTestCycles(3);

    Serial.println("Exercise completed");
}

void loop() {
}
```

A função `showTrackStatus()` recebe uma condição lógica. A função `showRoute()` recebe o número da rota. A função `runTestCycles()` recebe a quantidade de repetições.

As funções são chamadas em `setup()`, de modo que a sequência acontece uma vez a cada inicialização. O `delay(1000)` é apenas uma pausa inicial para este teste.

Exercício prático

Meta: conferir decisões e repetições no Monitor Serial, sem comandar a maquete.



1. Abra a Arduino IDE e crie um novo programa.
2. Copie o programa completo do item 9.
3. Salve com o nome `intermediate_02_decisions_loops`.
4. Conecte a ESP32 de bancada e selecione a placa e a porta corretas.
5. Utilize Verificar/Compilar e corrija eventuais erros.
6. Envie o programa para a ESP32.
7. Abra o Monitor Serial na velocidade de 115200 baud.
8. Pressione EN ou RESET para observar a sequência desde o início.

Resultado esperado:

```
EFMN 74 - Decisions and loops
Track occupied
Track clear
Route 2 selected
Test cycle: 1
Test cycle: 2
Test cycle: 3
Exercise completed
```

Alterações para praticar

1. Troque `showRoute(2)` por `showRoute(1)` e confira a mensagem.
2. Experimente `showRoute(5)`. O resultado deverá ser `Unknown route`.
3. Troque `runTestCycles(3)` por `runTestCycles(5)`. Confira os cinco números apresentados.
4. Experimente `runTestCycles(0)`. Nenhuma mensagem de ciclo deverá aparecer.
5. Acrescente uma opção `case 3` em `showRoute()`, com a mensagem `Route 3 selected` e um `break`. Depois teste com `showRoute(3)`.
6. Reescreva a repetição da função `runTestCycles()` utilizando `while`, mantendo o contador, a condição e o incremento. O resultado deverá permanecer igual.

Faça uma alteração de cada vez, compile, envie e confira o resultado.

10. Erros comuns



Erro	Como corrigir
Usar = ao tentar comparar valores.	Utilizar == para verificar igualdade.
Colocar um ponto e vírgula logo após if (...).	Retirar esse ponto e vírgula e colocar o bloco entre chaves.
Esquecer o incremento do contador no while.	Garantir que a condição possa mudar e a repetição termine.
Confundir < com <=.	Conferir se o valor limite deve participar da repetição.
Esquecer o break no switch.	Encerrar cada opção quando a intenção for executar apenas o caso escolhido.
Esperar uma mensagem única colocando a chamada em loop().	Usar setup() quando a tarefa deve ocorrer apenas na inicialização.

Resumo do capítulo

- if executa instruções quando uma condição é verdadeira.
- else define a alternativa quando a condição é falsa.
- else if permite verificar outras condições.
- Operadores lógicos combinam informações em uma decisão.
- switch escolhe entre valores inteiros previstos.
- for organiza repetições com um contador.
- while repete enquanto uma condição for verdadeira.
- do ... while executa o bloco pelo menos uma vez.
- As repetições precisam permitir que o programa avance para as próximas tarefas.

Próximo capítulo: Organização do programa em módulos.